Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

VeriNews: News Credibility and Verification System

Team Name: *Team #*	*ByteForce* *DSCPP-III-2026-T224*
Team member 1 (Team Lead)	*Shambhavi Srivastava* *Planning / Design / Coordination* 
Team member 2	*Abhinav Panwar* *C++ / DSA / Development* 
Team member 3	*Akshat Sharma* *Testing / Documentation / Metrics* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Misinformation moves fast. People don't. And most readers don't have a simple way to decide whether a headline deserves even a little trust.That's the gap VeriNews tries to fill.You type in a headline or a short claim. The system tidies the text, pulls out the important keywords, then checks those against a stored set of news records. After that, it assigns a match-based score and drops the claim into one of three buckets: Likely Verified, Needs Further Verification, or Potentially Suspicious.No, VeriNews isn't a "truth machine." It doesn't pretend to prove reality. What it does do is flag what looks questionable (or strongly supported) and show the user what drove that output. The reasoning stays visible. That part matters.This project also doubles as a hands-on DSA + OOP build. Under the hood, we're using hashing, BSTs, linked lists, queues, plus searching and sorting techniques. The whole thing is organized through C++ classes so modules stay clean. Records and verification logs are file-backed, meaning the program keeps its memory between runs.The target is an explainable, compact prototype—useful as a first-pass credibility check and solid proof that DSA and OOP can actually team up on a real-world problem.

Updated Project Approach and Architecture

We're building VeriNews as a modular C++ console application. One module, one job. A main controller ties everything together.- User/Input Module: takes the headline/claim from the user - News Module: defines a news record (claim text, keywords, source details) - News Database Module: loads/saves the predefined records via file handling Then the pipeline kicks in.The News Analyzer cleans the claim and extracts keywords. After that, the Search Module uses a hash-driven keyword index to pull related records quickly. For ordered lookup, we lean on a Binary Search Tree, and we use sorting to rank/compare the candidate matches.Next comes the Verification Engine. It merges the match signals into a credibility score and translates that score into one of the three categories. Finally, the Report Module prints the result and stores a history of checks for later viewing.We're keeping the design class-based and encapsulated. Inheritance/polymorphism will be used where it genuinely fits instead of being forced. Implementation uses standard C++ tooling: vector, string, map, unordered$_map, queue, fstream, algorithm$.

Tasks Completed

Task Completed	Team Member
Problem and background study: Researched how misinformation spreads and why a preliminary verification tool is useful.	Shambhavi Srivastava
Requirement analysis: Defined what the system needs for input, processing, verification, results and reporting.	Shambhavi Srivastava
System architecture: Designed the complete flow, from claim input through keyword processing, searching and scoring to final classification.	Akshat Sharma
DSA mapping: Decided where linked lists, hashing, BST, queues, searching and sorting fit in the system.	Abhinav Panwar
OOP design: Planned the User, News, NewsDatabase, NewsAnalyzer, VerificationEngine and Report classes.	Abhinav Panwar
Data-storage design: Planned file-based storage for predefined news records and verification history.	Akshat Sharma

Challenges/Roadblocks

The first big question was: what should “verification” even mean here? If we acted like we could “detect truth,” that would be dishonest. So we narrowed it down to something practical and explainable—match, score, show your work.Then came the design translation problem. Turning messy text processing into clean data structures and OOP modules isn’t automatic. We had to connect keyword cleanup to hashing, searching, sorting, and BST organization without ending up with a tangled architecture.Classification was another headache. If the scoring thresholds don’t clearly separate strong matches from weak ones, the three-category output becomes meaningless.Implementation has its own traps too. File handling needs to be reliable. No accidental overwrites. No lost history. Input validation and menu handling also have to be tight, because console programs can fall apart fast with unexpected user input.Testing won’t be trivial either. The scoring logic needs test cases for exact matches, partial overlaps, totally unrelated claims, and vague inputs that don’t carry enough info to judge.

Tasks Pending

Task Pending	Team Member (to complete the task)
News and User Classes: Write the classes, constructors, data members and member functions.	Abhinav Panwar
Data Storage Module: Implement file handling for news records and verification history.	Akshat Sharma
Text Processing: Implement text cleaning and keyword extraction for input claims.	Abhinav Panwar
Hash/Search Module: Build the keyword index and the search for relevant records.	Abhinav Panwar
Verification Engine: Implement matching, scoring and category assignment.	Shambhavi Srivastava
Report/UI Module: Complete the console interface, result display and history/report handling.	Shambhavi Srivastava
Testing and Evaluation: Run the predefined test cases, record results and evaluate the system.	Akshat Sharma

Project Outcome/Deliverables

The end deliverable is a C++ console prototype. A user submits a headline/claim. The system gives a preliminary credibility assessment based on stored records, explains the match reasoning, and categorizes the claim accordingly.Each data structure has a clear job:- hashing + searching for fast retrieval - BST + sorting for ordered lookup and comparison - queues for managing processing flow - linked lists for record-chain management - file handling for persistence across sessions - classes for modular, encapsulated codeDeliverables include:- validated news/claim input - text cleaning + keyword extraction - predefined news record storage - hash/search-based retrieval - match scoring + credibility calculation - classification into Likely Verified / Needs Further Verification / Potentially Suspicious - report output + verification history - C++ source code - testing + evaluation with predefined cases.

Progress Overview

Progress Status Design-wise, the heavy lifting is done: background research, requirements, architecture, DSA mapping, class/module planning, and storage design are complete. The full workflow—input through final category—is clearly laid out.Now it's build time. The remaining work centers on implementing the C++ classes, file operations, keyword extraction, hash indexing + search, scoring rules, the verification engine, the UI flow, and then running the test plan with proper evaluation.One principle stays non-negotiable: VeriNews gives explainable preliminary results, not final verdicts. If there's no close match in stored records, the system should admit uncertainty instead of guessing.Later on, sure, this could expand—NLP/ML, live news feeds, fact-check APIs, maybe even a web/mobile UI. For this phase, though, the focus is straightforward: show DSA + OOP + file handling working together in a clear, usable verification pipeline.

Codebase Information

(Repository link, branch, and information about important commits.)

Repository: https://github.com/AbhinavPanwar5099/VeriNews

Branch: main

Important committed changes:

1. **Data structures and search module:** Text cleaning and keyword extraction, hash table keyword index with linked-list record chains, date-based BST, merge sort ranking of matches, and binary search for source lookup.
2. **Verification and report module:** Five-factor credibility scoring, override rules for debunked and unmatched claims, classification into Likely Verified, Needs Further Verification and Potentially Suspicious, and the console report with menus and input validation.
3. **Storage and testing module:** File saving and reloading of news records and verification history, FIFO queue for batch checking, undo stack for admin changes, and predefined test cases with accuracy and confusion matrix evaluation.

The repository contains the three modules supporting the Phase-II workflow from news claim input to credibility result display.

Testing and Validation Status

Test Type	Status	Notes
Requirement / Design Validation	Completed	Problem statement, requirements, architecture and DSA/OOP mapping have been reviewed.
Input Processing Testing	Pending	Waiting on the keyword extraction and text-cleaning implementation.
Search / Hash Testing	Pending	Hash-based indexing and record search are not yet implemented.
Matching / Scoring Testing	Pending	Exact and partial matching, scoring and classification need to be built first.
Verification Engine Testing	Pending	Category logic (Likely Verified / Needs Further Verification / Potentially Suspicious) is not yet implemented.
File Storage Testing	Pending	Saving and loading of records and history is still in development.
Main Program Testing	Pending	The menu-driven program will be tested once all modules are integrated.
Overall Integration Testing	Pending	Full integration and evaluation will follow module completion.

Deliverables Progress

Problem & Requirement Analysis	Completed
System Architecture	Completed
DSA / OOP Mapping	Completed
Class & Module Design	Completed
Data Storage Design	Completed
Text / Keyword Processing	In Progress
Hash/Search & Matching	In Progress
Verification Engine	Pending
Final Integration & Testing	Pending